

Viabilidade: 4º coletor via Docker SDK (cgroups)

TCC — Benchmark de monitoramento de VNF (eBPF · Sysstat · Prometheus) · levantamento de 06/10/2026
· prazo: ~20 dias

1. Veredito

Viável, com risco moderado. Implementar e rodar cabe com folga (~2 dias de código, ~3,5 h de

máquina). O gargalo é a **escrita**: o 4º coletor toca todos os capítulos, e a conclusão ainda não existe. Só

vale a pena se houver um ponto de decisão no **dia 4** e se a escrita correr em paralelo às coletas. Há um

risco técnico (item 3.1) que pode invalidar a comparação e precisa ser testado primeiro.

2. O que o repositório já oferece (reuso)

- Os observadores têm 110–180 linhas cada e seguem o mesmo contrato: UDP :9999 respondendo um JSON com os mesmos campos. Um observador_docker.py entra nesse molde sem mexer no WAF, no cliente ou no probe.py.
- O docker-compose.sysstat.yml serve de modelo. O WAF já tem container_name: waf-server, o que facilita consultar o container certo.
- run_multi.sh valida o nome da ferramenta pelo arquivo run_<tool>.sh. Basta criar run_docker.sh (cópia do sysstat) e um compose novo.
- compare.py tem a lista TOOLS fixa; é uma alteração de poucas linhas. Os 30 runs × 4 volumes das 3 ferramentas atuais já existem.
- A lib docker **não está instalada** no host (ModuleNotFoundError). Precisa entrar na imagem do coletor.

3. Riscos técnicos (por gravidade)

#	Risco	Impacto / mitigação
3.1	Latência da consulta. container.stats(stream=False) costuma bloquear cerca de 1 s, porque o daemon amostra a CPU duas vezes. (Comportamento conhecido; não medi aqui, validar no protótipo.)	A métrica primária é o RTT do UDP a cada 1 s. Se o coletor consultar o Docker a cada probe, o RTT será da ordem de segundos e não será comparável aos ~0,5 ms das outras. Alternativa: uma thread em segundo plano com stream=True e o UDP respondendo do cache. Funciona, mas muda o que o RTT mede (deixa de incluir o custo da coleta). Decisão metodológica a alinhar com o orientador.
3.2	Custo fora do processo. Quem lê o cgroup é o dockerd, não o observador.	CPU/memória do observador (collector_*) subestimam o custo real. Para ser honesto na comparação, medir também o dockerd

ou declarar a limitação.

Memória não é a mesma grandeza.		
3.3	Sysstat/eBPF/Prometheus usam RSS (psutil). O cgroup reporta uso com <i>page cache</i> ; o <code>docker stats</code> subtrai <code>inactive_file</code> .	Risco de diferença que é artefato da métrica. Documentar a fórmula e, se possível, ler <code>memory.stat</code> (campo anon) para aproximar do RSS.
3.4	Rede. WAF e observador usam <code>network_mode: host</code> ; as estatísticas de rede do Docker não refletem o WAF.	Bytes RX/TX é métrica de validação, não de comparação. Marcar como "não aplicável" nesse coletor ou manter a leitura de <code>/proc/net/dev</code> .
3.5	Segurança/isolamento. Exige montar <code>/var/run/docker.sock</code> no observador (acesso equivalente a root no host).	Aceitável em ambiente de teste; registrar como ressalva na metodologia.
3.6	Kernel mudou. As coletas foram feitas em 7.0.0-15; o host está em 7.0.0-38.	Um 4º coletor medido em kernel diferente ameaça a validade. Ver estratégia de execução (seção 5).
3.7	CPU: o Docker calcula % relativo ao total de CPUs do host; o psutil, relativo a 1 núcleo.	Conversão simples ($\times$ nº de CPUs), mas precisa estar escrita e testada.

4. Estimativa de esforço

Frente	Estimativa	Observação
Observador + <code>compose + run_docker.sh + compare.py</code>	1–2 dias	Inclui validar o item 3.1 e a conta de CPU/memória.
Gráficos	0,5 dia	As 4 figuras de docs/imagens precisam da 4ª série. Não achei o script gerador no repositório; confirmar onde ele está.
Coleta (30 runs $\times$ N = 100k, 500k, 1M, 2M)	~3–3,5 h por ferramenta	Pelo <code>duration_s</code> medido (26, 53, 86, 153 s) + ~15 s de overhead por run. Roda à noite, sem intervenção.
Texto: Introdução, Revisão, Metodologia, Resultados	3–4 dias	Ver lista abaixo.
Conclusão (ainda não escrita)	1–2 dias	<code>main.tex</code> tem a linha de Considerações Finais comentada.
Slides e README	1 dia	3 .pptx em docs/apresentacao/ + docs/*.md.

Onde o texto precisa mudar (menções a "três" abordagens/coletores: Introdução 3, Revisão 7, Metodologia 13, Resultados 10):

- **Introdução:** problema, hipótese, objetivo geral e específicos ("implementar três monitores").
- **Revisão:** nova seção sobre cgroups e a API do Docker; parágrafos de comparação das "três abordagens"; tabela de trabalhos relacionados.
- **Metodologia:** subseção do coletor, topologia/C4, métricas, limitações (3.2–3.5).

- **Resultados:** as 6 seções, 4 figuras, tabela-síntese; as comparações par a par passam de 3 para 6 pares.
- Resumo/abstract, conclusão, slides e `relatorio_projeto.md`.

5. Estratégia de execução recomendada

1. **Dias 1–4:** protótipo e teste curto (5 runs, $N = 100k$, - - pre). **Ponto de decisão no dia 4:** se o item 3.1 não tiver solução defensável, o coletor vira prova de conceito.
2. **Controle do kernel:** reexecutar Sysstat em $N = 100k$ e $1M$ (30 runs, ~ 1 h) no kernel atual. Se bater com os dados antigos dentro do IC95%, só o Docker precisa de coleta nova. Se divergir, reexecutar tudo (~ 13 h de máquina, cerca de 3 noites).
3. **Dias 5–8:** coleta oficial do 4º coletor, à noite. Durante o dia, escrever a Revisão e a Metodologia (independentemente dos números).
4. **Dias 9–14:** Resultados com os dados finais, Introdução e Conclusão.
5. **Dias 15–18:** revisão com o orientador e ajustes. **Dias 19–20:** folga.

Plano B se o prazo apertar: manter as 3 ferramentas como resultado principal e incluir o Docker como "estudo complementar" ($N = 100k$ e $1M$, 30 runs), em uma seção separada. Evita reescrever a síntese inteira.

6. Perguntas para o orientador antes de começar

- O RTT do coletor Docker deve incluir a consulta ao daemon (item 3.1) ou usar leitura em segundo plano?
- O custo do `dockerd` deve entrar na comparação (item 3.2)?
- O 4º coletor entra em todas as análises ou como estudo complementar (plano B)?

Estimativas de esforço são minhas, a partir do tamanho do código e dos tempos medidos nos JSONs de resultado; não foram validadas com um protótipo.